

Name : Dinesh V

Slot : C

Reg. No. : 192524309

Date : 18.07.2026

Day : Saturday

Course : Compiler Design

CO1 - AT1

Code : CSA1401

1. Trace the process of Input exp. $P = (a * b) + (b * c) * 2$ through all phases of a compiler.

(a) Identify & list all tokens generated during Lexical Analysis.

(b) Construct the syntax tree during the parsing phase.

(c) Generate the Intermediate code for the given expression.

(d) Apply suitable optimizations to the Intermediate code.

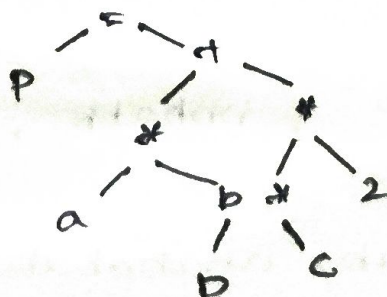
(e) Explain the significance of final target code produced.

(a)

Lexeme	Token	Lexeme	Token	Lexeme	Token
P	Identifier	b	Identifier	*	Mul. Operator
=	Assignment operator	)	Right Parenthesis	c	Identifier
(	Left parenthesis	+	Add. Operator	)	Right Parenthesis
a	Identifier	(	Left parenthesis	*	Mul. Operator
*	Mul. Operator	b	Identifier	2	Integer constant

Total tokens = 15

(b) Syntax Tree:



(c) Intermediate Code:

$t_1 = a * b$

$t_2 = b * c$

$t_3 = t_2 * 2$

$t_4 = t_1 + t_3$

$p = t_4$

...

(d) Optimized Intermediate Code:

$t_1 = a * b$

$t_2 = b * c$

$p = t_1 + (t_2 * 2)$

(e) Significance of Final Target Code:

→ Machine dependent code generated by the compiler.

→ Optimized for efficient CPU execution.

→ Reduces execution time & memory usage.

→ Executable by the process.

2. For each of the following Regular expressions, construct the language. Write any five valid strings for each regular expression.

(a) $(aa^*)^*$

ϵ , a, aa, aaa, aaaa.

(b) $(b^*)^* abb$ (c) $(a^* b^*)^* aba (a^* + b^*)^*$

aba, aaba, abaa, abab, aaababb

(b) $(b^*)^* abb$

abb, babb, bbabb, bbbabb, bbbbabb

(d) $(ab)^*$

ϵ , ab, abab, abbbab, abababab

(e) $(aba)^* + (a^*b^*)^*$

ϵ , aba, abaaba, aabb, aaabbb

(i) Find the no of tokens in the following C statement

is

```
main()
```

```
{
```

```
int a = 10
```

```
char b = "abc";
```

```
int c = 30;
```

```
char d = "xyz";
```

```
in /* comment */ t m = 40.5;
```

```
}
```

Statement	Token
main()	4
int a = 10	5
char b = "abc";	5
int c = 30 ;	5
char d = "xyz";	5
In /comment /tm=40.5	5

(ii) Identify the lexical errors in the following C statement is

```
void main()
```

```
{
```

```
int x = 10, y = 20;
```

```
char *a ;
```

```
a = &x;
```

```
x = 1xab;
```

```
}
```

(1) 12ab is an Individual hexadecimal Constant.

(2) Hexadecimal should begin with 0x.

(3) Correct form :

$$x = 0xAB;$$

(4) Find a regular expression corresponding to each of the following subsets of $\{1\}^*$:

(i) The Language of all strings that have an even number of 0's.

$$(1^* 01^* 01^*)^* 1^*$$

(ii) The Language of all strings that contain at least one 1.

$$0^* 1 0^*$$

$$(0+1)^* 1 (0+1)^*$$

(iii) The Language of all strings in which both the no. of 0's and the no. of 1's are odd.

$$1^* (01^* 01^*)^* 01^*$$

(iv) The Language of all strings that start with 1 and end with 0.

$$1(0+1)^* 0$$

(v) The Language of all strings that do not contain consecutive 1's.

$$(0+10)^* 1$$

A lexical analyzer needs to tokenize (tokenize) the following input string:
 int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return }

Questions:

(a) How many tokens are there in given input strings?

Token	Count
Keywords	4
Identifiers	4
Operator	3
Integer Literal	1
Floating Literal	1
Separator	10

Total Tokens = 23

(b) Provide a list of tokens categorized by their type

Keywords: int, float, if, return

Identifiers: var123, var456, var123, var456

Operators: =, =, ==

Integer Literal: 12345

Floating Literal: 123.45

Separators: ; ; () { } ; , ()

(c) Assuming a finite state machine (FSM) & using lexical analysis, estimate the no. of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

Approximate Transitions:

Keywords $\rightarrow 20$

Identifiers $\rightarrow 28$

Operators $\rightarrow 6$

Integer Literal $\rightarrow 5$

Floating Literal $\rightarrow 8$

Separators $\rightarrow 10$

Estimated Total FSM state Transitions ≈ 77

Reason:

Each character read causes a state transition in the finite state machine until a complete token is recognized.